

SiloGuard

MVP de alerta temprana de deterioro de granos para productores PyME

Documentación final del trabajo práctico integrador

Resistencia, Chaco · 2026 · Programación III

Proyecto	SiloGuard
Tipo de entrega	Documentación final de MVP (Producto y Frontend)
Repositorio analizado	https://github.com/juanmac12/siloguard-app
Pantallas / prototipos	En el mockup de Figma y dentro del repositorio (src/design-system/ + componentes React Native)
Grupo e Integrantes	Grupo 22 - Cantero Juan Manuel - Escobar Alabert Lucas

Índice

1. Resumen ejecutivo
2. Problema detectado
3. Objetivo del MVP
4. Usuarios principales
5. Solución propuesta
6. Alcance de diseño y design system
7. Flujo de usuario end-to-end
8. Mapa de pantallas del MVP
9. Funcionalidades implementadas
- 10.Stack tecnológico y estructura del proyecto
- 11.Alcance actual, pendientes y fuera de alcance
- 12.Conclusión

1. Resumen ejecutivo

SiloGuard es una aplicación móvil que permite a productores agropecuarios PyME **monitorear el estado de sus granos almacenados y recibir una alerta con al menos 48 horas de anticipación** antes de que el grano empiece a deteriorarse, sin tener que abrir el silo ni depender de inspecciones que llegan siempre tarde. Una lanza con sensores (IoT) mide CO₂, temperatura y humedad en continuo, y la app traduce esas señales en información accionable: estado del silo (verde/amarillo/rojo), alertas en lenguaje simple con acción recomendada, historial gráfico para confirmar que una intervención funcionó, y un Pasaporte de Calidad para certificar cada lote.

El frontend es una app mobile construida con **React Native y Expo (Expo Router)**. Incluye autenticación (registro, login, verificación de email, recuperación), onboarding, dashboard multi-silo, detalle del silo con pronóstico, historial de sensores, sistema de alertas con resolución, gestión de silos (ABM), configuración de umbrales, Pasaporte de Calidad, perfil y estados especiales de conexión. Consume una **API REST propia (.NET 10 + PostgreSQL)** con autenticación JWT. El diseño se entrega **codeado** en el propio repositorio (design system con tokens y componentes reutilizables), no solo como prototipo estático.

2. Problema detectado

En Argentina se almacenan cerca de **100 millones de toneladas de granos por año**. Los productores del Litoral (Santa Fe, Entre Ríos, Chaco, Corrientes) guardan su cosecha en silos o silobolsas durante meses sin ningún sistema que les informe qué ocurre dentro. La humedad relativa de la región supera el 80% en los meses de mayor calor (noviembre a marzo), creando condiciones ideales para que hongos e insectos activen procesos de fermentación que degradan el grano desde adentro.

El deterioro es **invisible en sus primeras etapas**. Cuando el productor lo detecta de forma manual —abriendo el silo o notando olor y cambio de color— ya perdió entre el 20% y el 40% del lote. La fricción central es el **desfasaje entre el momento en que el problema empieza y el momento en que el productor lo descubre**.

Indicador	Valor
Grano almacenado por año en Argentina	~100 millones de toneladas

Pérdida estimada por deterioro	5 % - 10 % del total almacenado
Equivalente económico anual	USD 2.000 - 4.000 millones
Productores PyME en el Litoral sin solución	~40.000
Meses de mayor riesgo	Noviembre a Marzo
Humedad relativa ambiente (verano Litoral)	> 80 % sostenida
Tiempo desde inicio de fermentación a pérdida irreversible	48 - 72 horas

3. Objetivo del MVP

El MVP responde una sola pregunta: **¿el productor actúa sobre una alerta temprana con suficiente anticipación para salvar el grano?** Todo lo que no sirva para probar esa hipótesis queda fuera del MVP.

- Que el productor pueda registrarse, vincular un silo y ver su estado de un vistazo.
- Que reciba una alerta clara cuando una variable cruza un umbral, con una acción concreta recomendada.
- Que pueda revisar el historial de sensores para confirmar que su acción corrigió el problema.
- Que pueda certificar la calidad del lote (Pasaporte) y configurar los umbrales de su silo.
- Que el equipo valide el comportamiento del productor ante la alerta antes de sumar hardware masivo o funcionalidades avanzadas.

4. Usuarios principales

Usuario	Perfil	Necesidad principal	Acciones dentro de la app
Productor (rol principal)	Productor agropecuario PyME del Litoral. Ej.: Carlos, 48 años, Rafaela (Santa Fe), 550 ha, almacena 200-400 t de soja y 80-120 t de maíz.	Saber qué pasa dentro del silo sin inspeccionarlo, y ser avisado a tiempo para	Vincular silos, ver estado y valores, recibir y resolver alertas, revisar historial, configurar umbrales, certificar lotes,

	Maneja apps simples, sin formación técnica.	actuar.	contactar al técnico.
Administrador (rol de sistema)	Rol interno de SiloGuard para gestión.	Supervisar usuarios del sistema.	Acceso a listados protegidos por rol (demuestra autorización).

El MVP se enfoca en el **productor**; el rol Admin existe para demostrar autorización por roles (ver documentación de backend). No hay múltiples roles de negocio como en un marketplace: SiloGuard es una herramienta de un solo lado.

5. Solución propuesta

SiloGuard conecta una **lanza IoT** clavada en el silo con una **app móvil** que convierte las lecturas en decisiones. El flujo de valor:

1. El productor vincula un silo (nombre, tipo de almacenamiento, grano, tonelaje).
2. La lanza mide CO₂, temperatura y humedad en continuo; la app muestra el **estado del silo** (score 1-100 + semáforo) y los valores actuales.
3. Cuando una variable cruza el **umbral** configurado, se genera una **alerta** (Crítica / Advertencia) con descripción en lenguaje simple: qué pasa, dónde, cuánto tiempo queda y qué hacer.
4. El productor ejecuta la acción recomendada (airear, inspeccionar, contactar técnico) y **marca la alerta como resuelta** con una nota.
5. En el **historial** confirma que los valores mejoraron tras su intervención.
6. Al cerrar un ciclo de almacenamiento, el **Pasaporte de Calidad** certifica el lote (score, promedios, días monitoreados) con un código verificable para compartir con bancos, acopios o compradores.

6. Alcance de diseño y design system

A diferencia de un proyecto que entrega los prototipos, pantallas y sistema de diseño únicamente en Figma, SiloGuard entrega su diseño **codeado dentro del repositorio**, en dos capas:

- **Design system** (`src/design-system/`): prototipo HTML de referencia + **tokens** (paleta,

tipografía, grid de 8pt, radios, sombras) y un bundle de componentes. Es la fuente de verdad visual del producto.

- **Componentes React Native reutilizables** (`src/components/`, 17 en total): `Button`,

`Input`, `AlertCard`, `SensorStat`, `StatusBadge`, `ScoreRing`, `ListItem`, `NavBar`, `AuthHeader`, `EmptyState`,

`OfflineBanner`, `DeviceOfflineBanner`, `TutorialCard`, `Icon` (set de íconos unificado), `AuthStepDots` y `OnboardingStepProgress` (progreso de flujos por pasos) y `DisabledHint` — implementan el design system en la app real y se exponen mediante un barrel (`index.ts`).

La app incluye además una pantalla `/ds-showcase`: un catálogo navegable de los componentes del design system, que permite verlos todos renderizados en el dispositivo real.

Fundamentos de diseño: dark-first, español rioplatense (voseo), sistema de espaciado 8pt, íconos unificados en un único set (sin mezclar librerías), y estados visuales de color consistentes (ok / warn / critical). Cada pantalla contempla sus estados: carga, vacío, error y offline.

7. Flujo de usuario end-to-end

Usuario nuevo (onboarding + primera acción):

Descarga → Splash → Welcome → Registro (2 pasos) → Verificación de email
 → Permisos push → (vinculación de lanza, simulada) → Asignación del silo
 → Tutorial (1 vez) → Dashboard → 1ª acción: revisa el estado del silo ✓

Usuario recurrente (revisión diaria):

Splash → (sesión activa) → Dashboard → ¿algún silo en amarillo/rojo?
 → Detalle del silo → Historial y/o Detalle de alerta → resolver

Tarea crítica (responder una alerta):

Notificación / Lista de alertas → Detalle de alerta (qué pasa · dónde · cuánto tiempo · qué hacer)
 → (opcional) Historial o Contactar técnico
 → Marcar como resuelta (con nota) → confirmación → vuelve a la lista

8. Mapa de pantallas del MVP

La navegación se organiza en bloques, con una tab bar inferior de 4 secciones (Dashboard · Alertas · Calidad · Perfil). Estado real de cada pantalla en la app (rama `main`, conectada a la API):

Bloque	Pantalla	Descripción	Estado
Autenticación	Splash	Logo animado + verificación de sesión, decide destino	Implementada

Autenticación	Welcome	Propuesta de valor + CTAs registrarse / iniciar sesión	Implementada
Autenticación	Login	Email + contraseña, con validación y manejo de errores	Implementada (API real)
Autenticación	Registro (2 pasos)	Datos personales + establecimiento (Firebase + API)	Implementada
Autenticación	Verificación de email	Espera post-registro con reenvío	Implementada
Autenticación	Recuperar contraseña	Solicitud de reset por email	Implementada
Onboarding	Permisos push	Explicación previa + solicitud de permiso	Implementada
Onboarding	Tutorial walkthrough	Overlay de pasos, una sola vez	Implementada
Monitoreo	Dashboard	Lista de silos con estado, valores y banner de alertas	Implementada (API real)
Monitoreo	Detalle del silo	Score, 3 sensores, pronóstico 3 días, sparkline, menú ...	Implementada (API real)
Monitoreo	Historial de sensores	Gráfico por variable, rangos 24h-7d, paginado real	Implementada (API real)
Alertas	Lista de alertas	Solapas Todas/Críticas/Advertencias/Resueltas (filtro en la API)	Implementada (API real)

Alertas	Detalle de alerta	Qué/dónde/cuánto/qué hacer + resolver con nota	Implementada (API real)
Gestión de silos	Agregar silo	Alta con datos + lectura inicial (transacción). Vinculación de lanza simulada	Implementada (API real)
Gestión de silos	Editar / Eliminar silo	Edición (con selector de fecha nativo) y baja con confirmación	Implementada (API real)
Perfil	Mi perfil	Datos, accesos a configuración, cierre de sesión	Implementada (API real)
Perfil	Editar perfil	Nombre + establecimiento	Implementada (API real)
Perfil	Cambiar contraseña	Re-autenticación con contraseña actual	Implementada (API real)
Perfil	Notificaciones	Toggles de advertencias y silencio nocturno	Implementada (UI; preferencias disponibles en la API)
Perfil	Dispositivos (Mis lanzas)	Lista de lanzas vinculadas y su estado	Implementada (datos mock — sin entidad IoT en backend)
Umbrales	Selector de silo	Elegir el silo a configurar	Implementada (API real)
Umbrales	Editor de	Sliders CO ₂ /temp/humedad, guardar	Implementada

	umbrales	y restaurar recomendados	(API real)
Pasaporte	Lista de lotes	Lotes de la temporada con estado, score y días	Implementada (API real)
Pasaporte	Detalle del lote	Certificado con score, QR y compartir	Implementada (API real)
Soporte	Contacto con técnico	Llamada real (tel:) y mensaje por WhatsApp con contexto de la alerta precargado	Implementada (acciones reales vía Linking)
Interno	Showcase del design system (/ds-showcase)	Catálogo navegable de los componentes renderizados en el dispositivo	Implementada (herramienta de desarrollo)

Además, los **estados especiales** (celular sin conexión y lanza sin respuesta) se resuelven como banners globales reutilizables (`OfflineBanner`, `DeviceOfflineBanner`), no como pantallas separadas. Ambos componentes están contruidos y montados en Dashboard y Detalle del silo, pero hoy se activan mediante *flags* de simulación (`MOCK_IS_OFFLINE`, `MOCK_DEVICE_OFFLINE`, ambos en `false`): permiten demostrar el estado forzando el flag, pero todavía no están conectados a la detección real de conectividad ni al *heartbeat* de la lanza (ver 11.2).

9. Funcionalidades implementadas

Funcionalidad	Descripción
Autenticación y sesión	Registro (Firebase + API), login con JWT propio, verificación de email, recuperación y sesión persistente (expo-secure-store)
Onboarding	Permisos push con explicación previa y tutorial walkthrough de primer ingreso
Dashboard multi-silo	Lista de silos con estado (verde/amarillo/rojo), valores

	resumidos y banner de alertas activas
Detalle del silo	Score 1-100, tiles de CO ₂ /temp/humedad, pronóstico de 3 días y sparkline de 7 días
Historial de sensores	Gráfico por variable con selector de rango y paginado real contra la base (Skip/Take, "Cargar más")
Sistema de alertas	Lista filtrable por 2 parámetros resueltos en la API, detalle en lenguaje simple y resolución con nota
Gestión de silos (ABM)	Alta (con lectura inicial transaccional), edición y baja con confirmación
Configuración de umbrales	ABM por silo con guardado transaccional y restaurar valores recomendados
Pasaporte de Calidad	Lista de lotes, certificado con score histórico, código de verificación y compartir (copiar link real)
Perfil y preferencias	Datos del productor y establecimiento, cambio de contraseña, preferencias de notificación
Contacto con técnico	Desde una alerta: llamada directa (tel:) o mensaje de WhatsApp con el contexto de la alerta precargado (Linking)
Estados especiales	Banners reutilizables de sin conexión y lanza sin respuesta, con datos cacheados y acciones deshabilitadas (hoy accionados por flags de simulación)
Feedback y usabilidad	Estados de carga, validación en cliente, mensajes de error claros y confirmaciones

10. Stack tecnológico y estructura del proyecto

Capa / herramienta	Versión	Uso dentro del proyecto
React Native + Expo	RN 0.81.5 · Expo SDK 54	Base de la app móvil, ejecución en Android/iOS vía Expo Go
React	19.1.0	Librería de UI subyacente
TypeScript	5.9.2	Tipado de silos, alertas, lotes, perfil y formularios
Expo Router	6.0.24	Navegación basada en archivos (src/app/), con grupo de rutas (tabs)
React Context (AppDataContext)	—	Estado global de datos y acciones; valida el token contra la API al iniciar
expo-secure-store	15.0.8	Persistencia segura del JWT
Firebase Auth (SDK cliente)	12.15.0	Registro y verificación de email
@react-native-community/datetimepicker	8.4.4	Selector de fecha nativo (fecha de acopio, en edición de silo)
expo-clipboard	8.0.8	Copiar link de verificación del pasaporte
API backend .NET	—	El frontend consume endpoints REST con Authorization: Bearer

Estructura general (src/):

- app/ — pantallas por rutas: raíz (splash, welcome, login, register...), (tabs)/ (dashboard, alertas, pasaporte, perfil), y subrutas (silo/[id], alerta/[id], umbrales/[siloId], perfil/*).

- `components/` — componentes reutilizables del design system (Button, Input, AlertCard, SensorStat, StatusBadge, ScoreRing, banners...).
- `services/` — clientes HTTP por dominio (`authApi`, `silosApi`, `alertaApi`, `loteApi`, `perfilApi`, `umbralApi`) sobre `apiFetch`, más `tokenStorage`.
- `contexts/` — `AppDataContext` (datos + acciones) y `ThemeContext` (tema dark-first).
- `design-system/` — tokens, prototipo HTML y bundle de componentes (fuente de verdad visual).
- `config/`, `constants/`, `utils/` — configuración de API/Firebase, tema y utilidades.

Aclaración técnica: las lecturas provienen de la API (la lanza IoT está simulada por el seeder del backend); el refresco se resuelve al abrir/recargar, suficiente para el MVP.

11. Alcance actual, pendientes y fuera de alcance

11.1 Alcance actual implementado

- Flujo completo del productor: auth, onboarding, monitoreo, alertas, gestión de silos, umbrales, pasaporte y perfil.
- Consumo real de la API con JWT; filtros y paginado resueltos en el servidor.
- Design system codeado (tokens + componentes) y estados de carga/vacío/error/offline.
- Pasaporte de Calidad con certificado y compartición.

11.2 Funcionalidades parciales o dependientes

- **Pantalla de Notificaciones:** la UI está lista (toggles de advertencias y silencio nocturno); la persistencia de preferencias ya existe en la API (`GET/PUT /api/perfil/notificaciones`) pero la pantalla todavía usa estado local y no la consume.
- **Compartir pasaporte como PDF/imagen:** hoy muestra "Próximamente" (copiar link sí funciona); requiere captura de vista / generación de PDF en el cliente.
- **Dispositivos / "Mis lanzas":** usa datos mock — no hay entidad de dispositivo IoT en el backend (no lo exige el alcance).
- **Vinculación de lanza (QR + WiFi):** representada de forma simulada dentro del alta de silo; el escaneo de redes usa un listado fijo y la lectura inicial del silo se genera con valores aleatorios dentro de rango. No hay hardware real en el MVP.
- **Banners de estado especial:** `OfflineBanner` y `DeviceOfflineBanner` están implementados y montados, pero se accionan con *flags* de simulación (`MOCK_IS_OFFLINE`, `MOCK_DEVICE_OFFLINE`); falta conectarlos a la detección real de red y al *heartbeat* de la lanza.

- **Selector de fecha nativo:** disponible en la edición de silo; en el alta la fecha de acopio se autocompleta con la fecha del día sin selector.
- **Pull-to-refresh:** el `AppDataContext` ya expone `refresh()` y las pantallas lo invocan al montarse, pero todavía no está enganchado a un `RefreshControl` en los listados.

11.3 Fuera del alcance actual del MVP

- Envío real de notificaciones push (Expo/FCM) y evaluación de lecturas en segundo plano.
- Ingesta de lecturas desde hardware IoT real (hoy simuladas por el seeder).
- Resumen semanal automático, pronóstico meteorológico con API externa real.
- Multi-establecimiento y roles de negocio adicionales (técnico/agróonomo con cuenta).
- Login social real con Google/Apple (los botones quedan como placeholder de UI).

12. Conclusión

SiloGuard cumple con las condiciones de un MVP porque se concentra en validar un problema concreto y medible: **si el productor actúa a tiempo sobre una alerta temprana para salvar el grano**. La app no intenta resolver toda la cadena agroindustrial desde el inicio; prioriza el flujo principal de monitoreo, alerta, acción y certificación.

El proyecto cuenta con un usuario claramente definido, 25 pantallas funcionales organizadas por bloques (más un showcase interno del design system), formularios con validación, navegación completa, integración con autenticación propia, consumo de una API REST real, filtros y paginado resueltos en el servidor, y un design system codeado en el repositorio con 17 componentes reutilizables. Esto permite demostrar el valor central de la solución sin depender todavía de hardware masivo, envío de push o funcionalidades avanzadas.

Como entrega final, esta documentación de producto y frontend se complementa con la **documentación del backend** (docs/[DOCUMENTACION-FINAL-BACKEND.md](#)) y con todo lo hecho en [Figma](#) donde se muestran prototipos, pantallas y todo el sistema de diseño. En conjunto muestran tanto la experiencia propuesta para el productor como la implementación técnica que la sostiene.